МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Кафедра комп’ютерної інженерії

Дисципліна «Програмування»

Спеціальність 123 «Комп’ютерна інженерія»

І семестр

ЗВІТ

до лабораторної роботи №4

Модульне програмування, низькорівневі порозрядні операції, бітові маски та командний процес розробки через Pull Request у Git

Варіант 3

Виконав: студент групи КІ-26-1
Волохов Дмитрій

2026

1. Мета роботи та індивідуальні завдання

Опанувати поділ програми на функції, використання прототипів і локальних статичних змінних. Навчитися виконувати побітові операції над 32-бітним регістром, пакувати та відновлювати поля телеметрії. Закріпити роботу з окремою гілкою Git і передавання результатів через Pull Request.

Завдання 1. Модульність і static

Реалізувати функцію перевірки числа на простоту. Локальна статична змінна має рахувати, скільки простих чисел виявлено за сеанс роботи. Прототипи функцій розмістити перед main; аргументи обчислювальних функцій передавати за значенням.

Завдання 2. Регістр GPIO

У 32-бітному беззнаковому регістрі встановити біт 4, скинути біт 8, інвертувати біт 14 і перевірити стан біта 2. Вивести двійковий стан до та після кожної операції. Нумерація бітів починається з 0 у молодшому розряді.

Завдання 3. Пакування телеметрії

Розмістити в одному беззнаковому слові: пріоритет (0..15) у бітах 0..3; швидкість вентилятора (0..255) у бітах 4..11; код помилки (0..3) у бітах 12..13; живлення (0..1) у біті 14. Відновити поля масками та зсувами й перевірити їх збіг із початковими значеннями.

Засоби виконання

Мова C; компілятор GCC у середовищі MSYS2 UCRT64; редактор Visual Studio Code; система контролю версій Git. Робочий каталог: programming-sem1/lab04. Компіляцію виконано з параметрами -Wall -Wextra.

Алгоритм перевірки на простоту

Числа, менші за 2, не є простими. Для інших чисел перевіряються дільники від 2 до квадратного кореня з числа. Якщо залишок від ділення дорівнює 0, число складене. Умова divisor <= number / divisor рівнозначна перевірці divisor * divisor <= number для додатних чисел і запобігає переповненню під час множення.

Функція count_primes має локальну змінну static unsigned int total. Вона ініціалізується один раз, зберігає значення між викликами та збільшується лише для простого числа. Повторне введення простого числа враховується як нове виявлення. Після перезапуску програми лічильник знову починається з нуля.

2. Лістинг програми task1.c

```
#include <stdio.h>

int is_prime(int number);
unsigned int count_primes(int prime);

int main(void)
{
    int number;

    puts("Variant 3: prime numbers and static counter");
    puts("Enter q to finish.");

    while (1) {
        printf("Enter an integer: ");

        if (scanf("%d", &number) != 1) {
            break;
        }

        int prime = is_prime(number);
        unsigned int total = count_primes(prime);

        printf("%d: %s\n", number, prime ? "prime" : "not prime");
        printf("Primes found this session: %u\n\n", total);
    }

    return 0;
}

int is_prime(int number)
{
    if (number < 2) {
        return 0;
    }

    for (int divisor = 2; divisor <= number / divisor; ++divisor) {
        if (number % divisor == 0) {
            return 0;
        }
    }

    return 1;
}

unsigned int count_primes(int prime)
{
    static unsigned int total = 0;

    if (prime) {
        ++total;
    }

    return total;
}
```

3. Тестування першого завдання

```
gcc -Wall -Wextra task1.c -o task1
./task1
```

```
O9Odi@DESKTOP-1MKOJG3 UCRT64 ~/projects/programming-sem1/lab04
$ gcc -Wall -Wextra task1.c -o task1

O9Odi@DESKTOP-1MKOJG3 UCRT64 ~/projects/programming-sem1/lab04
$ ./task1
Variant 3: prime numbers and static counter
Enter q to finish.
Enter an integer: 1
1: not prime
Primes found this session: 0

Enter an integer: 2
2: prime
Primes found this session: 1

Enter an integer: 4
4: not prime
Primes found this session: 1

Enter an integer: 17
17: prime
Primes found this session: 2

Enter an integer: -5
-5: not prime
Primes found this session: 2

Enter an integer: q

O9Odi@DESKTOP-1MKOJG3 UCRT64 ~/projects/programming-sem1/lab04
$
```

Рисунок 1 - Перевірка чисел на простоту та робота статичного лічильника.

Число	Результат перевірки	Лічильник
1	Не є простим	0
2	Просте	1
4	Не є простим	1
17	Просте	2
-5	Не є простим	2

Під час сеансу виявлено два простих числа: 2 і 17. Для складеного числа 4, одиниці та від’ємного числа лічильник не змінюється. Введення q завершує цикл. Компілятор не вивів попереджень.

4. Маски та розрахунки для регістра GPIO

Для регістра використано uint32_t. Початкове значення обрано так, щоб біт 8 можна було скинути, а біт 2 залишався встановленим під час усіх модифікацій:

```
R0 = 0x00000104 = 260 = 2^8 + 2^2
BIN: 0000 0000 0000 0000 0000 0001 0000 0100
```

Встановлення біта 4

```
M4 = 1u << 4 = 0x00000010 = 16
R1 = R0 | M4
   = 0x00000104 | 0x00000010 = 0x00000114 = 276
```

Операція OR встановлює вибраний розряд в 1. Нулі маски залишають решту бітів без змін.

Скидання біта 8

```
M8  = 1u << 8 = 0x00000100 = 256
~M8 = 0xFFFFFEFF  (32 bits)
R2  = R1 & ~M8
    = 0x00000114 & 0xFFFFFEFF = 0x00000014 = 20
```

В інвертованій масці лише біт 8 дорівнює 0. Побітове AND скидає його, зберігаючи інші розряди.

Інвертування біта 14

```
M14 = 1u << 14 = 0x00004000 = 16384
R3  = R2 ^ M14
    = 0x00000014 ^ 0x00004000 = 0x00004014 = 16404
```

XOR з одиницею перемикає стан біта 14. XOR з нулем не змінює решту бітів.

Перевірка біта 2

```
M2 = 1u << 2 = 0x00000004
R3 & M2 = 0x00004014 & 0x00000004 = 0x00000004
check_bit(R3, 2) = 1
R_after_check = R3 = 0x00004014
```

Перевірка повертає 1, бо результат AND ненульовий. Регістр при цьому не змінюється. У функціях використано беззнакову одиницю UINT32_C(1); перевірка bit >= 32 запобігає зсуву на недопустиму кількість розрядів.

5. Лістинг програми task2.c

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

uint32_t set_bit(uint32_t value, unsigned int bit);
uint32_t clear_bit(uint32_t value, unsigned int bit);
uint32_t toggle_bit(uint32_t value, unsigned int bit);
int check_bit(uint32_t value, unsigned int bit);
void print_binary(uint32_t value);
void print_register(uint32_t value);

int main(void)
{
    uint32_t gpio = UINT32_C(0x00000104);

    puts("Variant 3: 32-bit GPIO register");

    puts("\nBefore set_bit(4):");
    print_register(gpio);
    gpio = set_bit(gpio, 4);
    puts("After set_bit(4):");
    print_register(gpio);

    puts("\nBefore clear_bit(8):");
    print_register(gpio);
    gpio = clear_bit(gpio, 8);
    puts("After clear_bit(8):");
    print_register(gpio);

    puts("\nBefore toggle_bit(14):");
    print_register(gpio);
    gpio = toggle_bit(gpio, 14);
    puts("After toggle_bit(14):");
    print_register(gpio);

    puts("\nBefore check_bit(2):");
    print_register(gpio);
    printf("Bit 2 = %d\n", check_bit(gpio, 2));
    puts("After check_bit(2):");
    print_register(gpio);

    return 0;
}
```

Лістинг task2.c (продовження)

```
uint32_t set_bit(uint32_t value, unsigned int bit)
{
    if (bit >= 32) {
        return value;
    }
    return value | (UINT32_C(1) << bit);
}

uint32_t clear_bit(uint32_t value, unsigned int bit)
{
    if (bit >= 32) {
        return value;
    }
    return value & ~(UINT32_C(1) << bit);
}

uint32_t toggle_bit(uint32_t value, unsigned int bit)
{
    if (bit >= 32) {
        return value;
    }
    return value ^ (UINT32_C(1) << bit);
}

int check_bit(uint32_t value, unsigned int bit)
{
    if (bit >= 32) {
        return 0;
    }
    return (value & (UINT32_C(1) << bit)) != 0;
}

void print_binary(uint32_t value)
{
    for (int bit = 31; bit >= 0; --bit) {
        putchar(((value >> bit) & UINT32_C(1)) ? '1' : '0');

        if (bit > 0 && bit % 4 == 0) {
            putchar(' ');
        }
    }
    putchar('\n');
}

void print_register(uint32_t value)
{
    printf("HEX: 0x%08" PRIX32 "\nBIN: ", value);
    print_binary(value);
}
```

6. Тестування другого завдання

```
O9Odi@DESKTOP-1MKOJG3 UCRT64 ~/projects/programming-sem1/lab04
$ gcc -Wall -Wextra task2.c -o task2

O9Odi@DESKTOP-1MKOJG3 UCRT64 ~/projects/programming-sem1/lab04
$ ./task2
Variant 3: 32-bit GPIO register

Before set_bit(4):
HEX: 0x00000104
BIN: 0000 0000 0000 0000 0000 0001 0000 0100
After set_bit(4):
HEX: 0x00000114
BIN: 0000 0000 0000 0000 0000 0001 0001 0100

Before clear_bit(8):
HEX: 0x00000114
BIN: 0000 0000 0000 0000 0000 0001 0001 0100
After clear_bit(8):
HEX: 0x00000014
BIN: 0000 0000 0000 0000 0000 0000 0001 0100

Before toggle_bit(14):
HEX: 0x00000014
BIN: 0000 0000 0000 0000 0000 0000 0001 0100
After toggle_bit(14):
HEX: 0x00004014
BIN: 0000 0000 0000 0000 0100 0000 0001 0100

Before check_bit(2):
HEX: 0x00004014
BIN: 0000 0000 0000 0000 0100 0000 0001 0100
Bit 2 = 1
After check_bit(2):
HEX: 0x00004014
BIN: 0000 0000 0000 0000 0100 0000 0001 0100

O9Odi@DESKTOP-1MKOJG3 UCRT64 ~/projects/programming-sem1/lab04
$
```

Рисунок 2 - Стан 32-бітного регістра до та після кожної операції.

Етап	HEX	Десяткове значення
Початковий стан	0x00000104	260
Встановити біт 4	0x00000114	276
Скинути біт 8	0x00000014	20
Інвертувати біт 14	0x00004014	16404
Перевірити біт 2	0x00004014	16404

Після модифікацій встановлені біти 14, 4 і 2. Кожна операція змінює лише цільовий біт; біт 2 зберігає стан 1. Функція print_binary виводить усі 32 біти групами по чотири.

7. Структура та розрахунок пакета телеметрії

Поле	Діапазон	Біти	Маска поля	Зсув
Пріоритет	0..15	0..3	0x000F	0
Вентилятор	0..255	4..11	0x0FF0	4
Код помилки	0..3	12..13	0x3000	12
Живлення	0..1	14	0x4000	14

Слово має тип uint16_t. Поля займають 15 бітів; старший біт 15 залишається нульовим. Маски полів не перетинаються. Перед пакуванням програма перевіряє допустимість введених значень.

Формула пакування

```
packet = (priority & 0x0F)
       | ((fan_speed & 0xFF) << 4)
       | ((error_code & 0x03) << 12)
       | ((power & 0x01) << 14);
```

Приклад: пріоритет 5, вентилятор 180, помилка 2, живлення 1

```
5   = 0x05;   5   << 0  = 0x0005
180 = 0xB4;   180 << 4  = 0x0B40 = 2880
2   = 0x02;   2   << 12 = 0x2000 = 8192
1   = 0x01;   1   << 14 = 0x4000 = 16384

packet = 0x0005 | 0x0B40 | 0x2000 | 0x4000
       = 0x6B45
       = 5 + 2880 + 8192 + 16384 = 27461
BIN:     0110 1011 0100 0101
```

Відновлення початкових полів

```
priority   = 0x6B45 & 0x0F         = 5
fan_speed  = (0x6B45 >> 4) & 0xFF  = 180
error_code = (0x6B45 >> 12) & 0x03 = 2
power      = (0x6B45 >> 14) & 0x01 = 1
```

Правий зсув переносить вибране поле в молодші біти. Маска відкидає біти інших полів. Функція unpack_telemetry записує результати у змінні main через передані адреси; вимогу передавання аргументів за значенням у першому завданні виконано окремо.

8. Лістинг програми task3.c

```
#include <stdio.h>
#include <stdint.h>

uint16_t pack_telemetry(unsigned int priority, unsigned int fan_speed,
                        unsigned int error_code, unsigned int power);

void unpack_telemetry(uint16_t packet, unsigned int *priority,
                      unsigned int *fan_speed, unsigned int *error_code,
                      unsigned int *power);

int main(void)
{
    unsigned int priority, fan_speed, error_code, power;
    unsigned int restored_priority, restored_fan;
    unsigned int restored_error, restored_power;

    puts("Variant 3: telemetry packing");
    puts("Enter priority (0..15), fan speed (0..255),");
    puts("error code (0..3), power (0..1), separated by spaces:");

    if (scanf("%u %u %u %u", &priority, &fan_speed,
              &error_code, &power) != 4) {
        puts("Input error: enter four integers.");
        return 1;
    }

    if (priority > 15 || fan_speed > 255 ||
        error_code > 3 || power > 1) {
        puts("Input error: a value is outside the allowed range.");
        return 1;
    }

    uint16_t packet = pack_telemetry(priority, fan_speed,
                                     error_code, power);

    unpack_telemetry(packet, &restored_priority, &restored_fan,
                     &restored_error, &restored_power);

    printf("\nOriginal: priority=%u, fan=%u, error=%u, power=%u\n",
           priority, fan_speed, error_code, power);

    printf("Packet: 0x%04X\n", (unsigned int)packet);

    printf("Restored: priority=%u, fan=%u, error=%u, power=%u\n",
           restored_priority, restored_fan,
           restored_error, restored_power);

    if (priority == restored_priority &&
        fan_speed == restored_fan &&
        error_code == restored_error &&
        power == restored_power) {
        puts("Verification: OK (all fields match).");
        return 0;
    }

    puts("Verification: FAILED.");
    return 1;
}
```

Лістинг task3.c (продовження). Контрольний запуск

```
uint16_t pack_telemetry(unsigned int priority, unsigned int fan_speed,
                        unsigned int error_code, unsigned int power)
{
    return (uint16_t)((priority & 0x0Fu) |
                      ((fan_speed & 0xFFu) << 4) |
                      ((error_code & 0x03u) << 12) |
                      ((power & 0x01u) << 14));
}

void unpack_telemetry(uint16_t packet, unsigned int *priority,
                      unsigned int *fan_speed, unsigned int *error_code,
                      unsigned int *power)
{
    *priority = packet & 0x0Fu;
    *fan_speed = (packet >> 4) & 0xFFu;
    *error_code = (packet >> 12) & 0x03u;
    *power = (packet >> 14) & 0x01u;
}
```

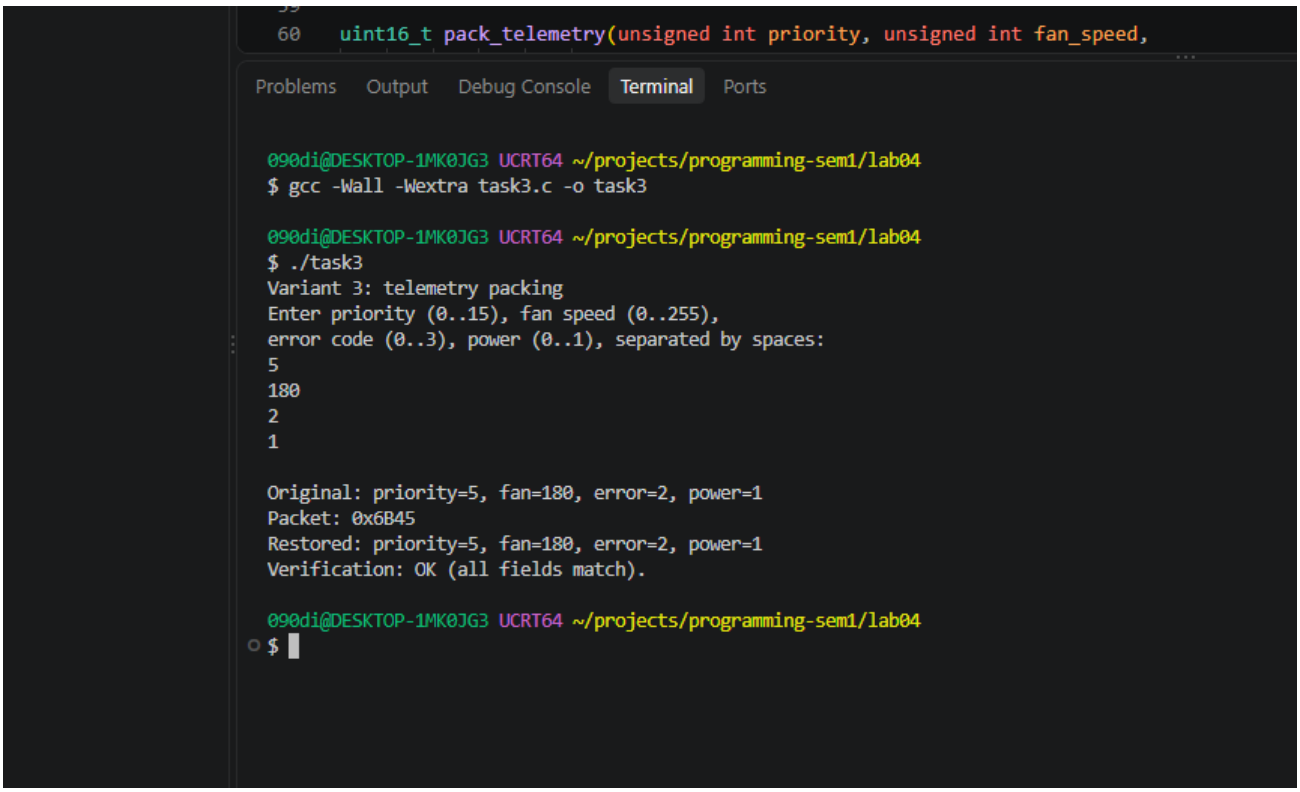


Рисунок 3 - Пакування та відновлення значень 5, 180, 2, 1.

У рядках Original і Restored наведено однакові параметри. Сформований пакет 0x6B45 відповідає ручному розрахунку. Повідомлення Verification: OK підтверджує збіг усіх чотирьох полів.

9. Перевірка граничних значень

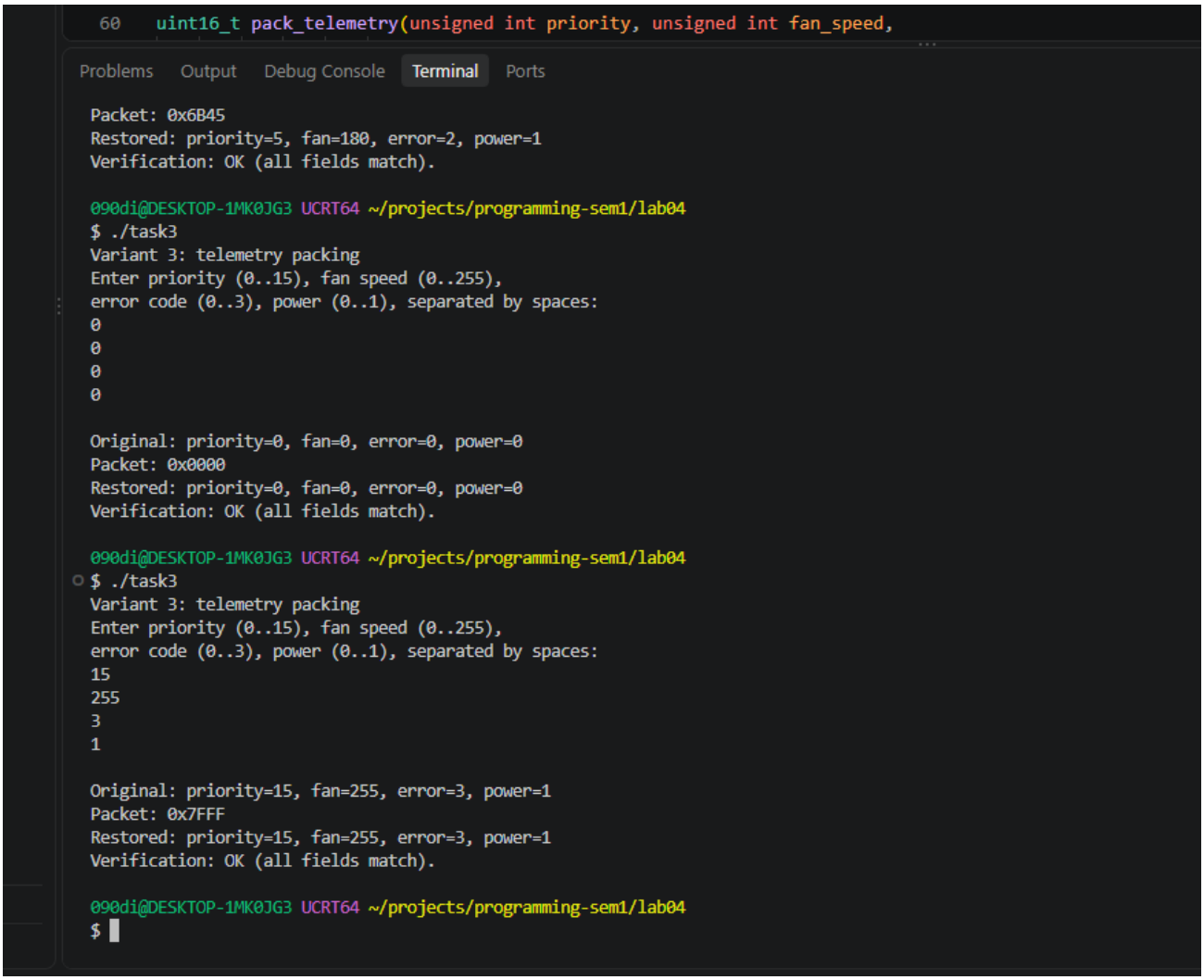


Рисунок 4 - Перевірка мінімальних і максимальних допустимих полів.

Вхідні поля	Пакет HEX	Пакет у 10 СЧ	Результат
0, 0, 0, 0	0x0000	0	OK
15, 255, 3, 1	0x7FFF	32767	OK

```
Pmin = 0 | (0 << 4) | (0 << 12) | (0 << 14) = 0
Pmax = 15 + 255 * 16 + 3 * 4096 + 1 * 16384
     = 15 + 4080 + 12288 + 16384 = 32767
     = 0x7FFF = 0111 1111 1111 1111
```

В обох граничних тестах відновлені значення збігаються з початковими. Навіть для максимальних полів біт 15 не встановлюється: усі дані вміщуються в задані 15 розрядів.

10. Робота з Git та висновки

Розробку виконано у гілці lab04 репозиторію programming-sem1. Для лабораторної створено каталог lab04 з файлами task1.c, task2.c і task3.c. Третю програму зафіксовано комітом 5ba5e32. Після коміту git status показав гілку lab04 і повідомлення working tree clean.

Додавання пояснювальної записки та передавання гілки

Звіт зберігається у каталозі lab04 під назвою Report_Lab04.pdf. Для включення звіту до гілки та передавання результатів використовуються команди:

```
git add Report_Lab04.pdf
git commit -m "docs(lab4): add mandatory engineering PDF report"
git push -u origin lab04
```

На GitHub створюється Pull Request із lab04 у main із заголовком Lab 4: Functions, Bitwise Operations and Register Emulation. В описі зазначаються мета, варіант 3 та виконані завдання. Pull Request дає змогу перевірити зміни перед включенням до основної гілки. Після перевірки або за вказівкою викладача виконується злиття; для здачі роботи потрібний статус Merged.

```
git switch main
git pull --ff-only origin main
```

Висновки

Під час роботи реалізовано три модульні програми мовою C. У першій програмі функція перевіряє число на простоту, а локальна статична змінна зберігає кількість знайдених простих чисел між викликами. Для послідовності 1, 2, 4, 17, -5 отримано два простих числа.

У другій програмі за допомогою масок виконано встановлення, скидання, інвертування та перевірку бітів 32-бітного регістра. Отримано кінцеве значення 0x00004014; стан біта 2 дорівнює 1. Цільові операції не змінюють сусідніх розрядів.

У третій програмі чотири параметри запаковано в 16-бітне слово й відновлено зсувами та масками. Для значень 5, 180, 2, 1 отримано пакет 0x6B45. Основний і два граничні тести підтвердили збіг полів після розпакування. Усі три програми скомпільовано з -Wall -Wextra без повідомлень про попередження.

Закріплено використання прототипів, передавання аргументів за значенням, локальної static-змінної, беззнакових типів фіксованої ширини та окремої робочої гілки Git.

11. Відповіді на контрольні запитання

1. Чому компілятору мови С необхідний прототип функції, якщо тіло підпрограми описане нижче у тому самому файлі?

Прототип заздалегідь повідомляє компілятору назву функції, тип її результату, кількість і типи параметрів. Коли компілятор обробляє виклик у main, він ще не дійшов до визначення функції нижче, тому використовує прототип для перевірки аргументів і правильного формування виклику. У сучасній мові С виклик функції без попереднього оголошення не допускається. Якщо визначення функції розташоване перед викликом, воно саме може виконувати роль такого оголошення.

```
int is_prime(int number);  /* prototype before main */
```

2. Що таке стековий кадр (Stack Frame) і які апаратні регістри процесора фіксують його межі під час виклику функції?

Стековий кадр - ділянка стека, пов’язана з одним викликом функції. Залежно від архітектури та способу компіляції вона може містити локальні змінні, збережені регістри, адресу повернення й аргументи, передані через стек. У типовому коді x86 регістр ESP указує на поточну вершину стека, а EBP - на опорну адресу кадру. У x86-64 відповідні регістри називаються RSP і RBP. Оптимізований код може не використовувати окремий покажчик кадру, а частину аргументів і локальних змінних зберігати в регістрах.

3. Чим фізично відрізняється локальна змінна зі специфікатором static від звичайної автоматичної локальної змінної на стеку?

Звичайна автоматична змінна існує протягом виконання свого блока. Для різних викликів функції створюються окремі її екземпляри; після завершення блока час життя закінчується. Зазвичай така змінна розміщується у стековому кадрі, хоча компілятор може використати регістр. Локальна static-змінна має одне постійне місце зберігання на весь час роботи програми, зазвичай у секції даних або .bss. Вона ініціалізується один раз і зберігає значення між викликами. Область видимості при цьому залишається локальною. У task1.c змінна total саме тому накопичує кількість простих чисел.

4. Чому спроба змінити значення змінної всередині функції через передачу за значенням (Pass-by-Value) не змінює оригінальну змінну в головній програмі?

Параметр функції отримує копію значення аргументу. Оригінальна змінна й параметр є різними об’єктами, тому присвоєння параметру змінює лише копію. Наприклад, якщо передати число 5 у функцію й присвоїти її параметру 10, початкова змінна залишиться рівною 5. Щоб отримати нове значення в main, можна повернути його через return і присвоїти змінній. Інший спосіб - передати адресу змінної та змінити об’єкт через покажчик; сам покажчик у С також передається за значенням.

Відповіді на контрольні запитання (продовження)

5. Як за допомогою побітових операцій зсуву та оператора «НІ» сформувати маску для гарантованого скидання обраного біта регістра в нуль?

Спочатку одиницю зсувають ліворуч на N позицій. Після інверсії отримують маску, у якій біт N дорівнює 0, а всі інші біти - 1. Побітове AND із цією маскою гарантовано обнуляє потрібний розряд і зберігає інші. Для uint32_t допустимі номери N від 0 до 31; одиниця має бути беззнаковою.

```
uint32_t mask = ~(UINT32_C(1) << N);
reg &= mask;
/* N = 8: mask = 0xFFFFFEFF */
```

6. Чому операцію перевірки стану прапорця виконують через побітове «І», а не через перевірку на рівність усьому числу?

AND із маскою виділяє потрібний біт незалежно від стану решти розрядів. Ненульовий результат означає, що прапорець установлено. Порівняння всього регістра з маскою перевіряє також усі інші біти й може повернути хибний результат. Наприклад, у регістрі 0x14 установлені біти 4 і 2: перевірка (reg & 0x10) != 0 істинна, але reg == 0x10 хибна через додатково встановлений біт 2.

```
int flag = (reg & (UINT32_C(1) << N)) != 0;
```

7. У чому полягає апаратна небезпека циклу Read-Modify-Write при роботі з регістрами мікроконтролера в умовах апаратних переривань?

Операція складається з читання регістра, зміни прочитаної копії та запису назад. Між читанням і записом може спрацювати переривання й змінити цей самий регістр. Основний код потім запише застарілу копію та втратить зміну обробника. Наприклад, основний код прочитав 0, переривання встановило біт 2 (регістр став 4), а основний код установив у своїй старій копії біт 4 і записав 16 замість потрібних 20. Для захисту застосовують апаратні атомарні регістри SET/CLEAR або критичну секцію з коректним збереженням і відновленням стану дозволу переривань. Специфікатор volatile сам по собі не робить Read-Modify-Write атомарним.

8. Що таке Pull Request у системі контролю версій Git і яку інженерну роль він відіграє в процесі рецензування коду (Code Review)?

Pull Request - пропозиція об’єднати зміни однієї гілки з іншою на платформі GitHub або аналогічному сервісі. Це механізм платформи для співпраці, а не окрема команда Git. У PR рецензент переглядає різницю між версіями, обговорює рішення, залишає зауваження та перевіряє результати автоматичних перевірок, якщо вони налаштовані. Автор може виправити код у тій самій гілці. Після перевірки зміни зливаються в цільову гілку. У цій роботі джерело змін - lab04, цільова гілка - main; статус Merged означає, що злиття вже виконано.